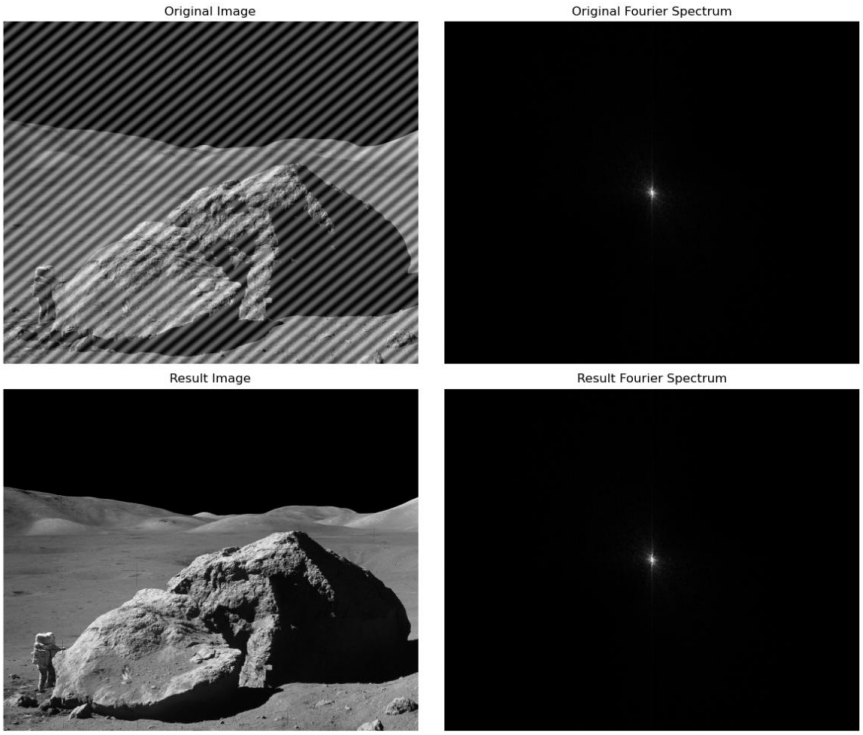
Name: 宋庭宇 Student ID: 314512091

1. Filtered Image of “astronaut-interference.tif”:



Source Code:

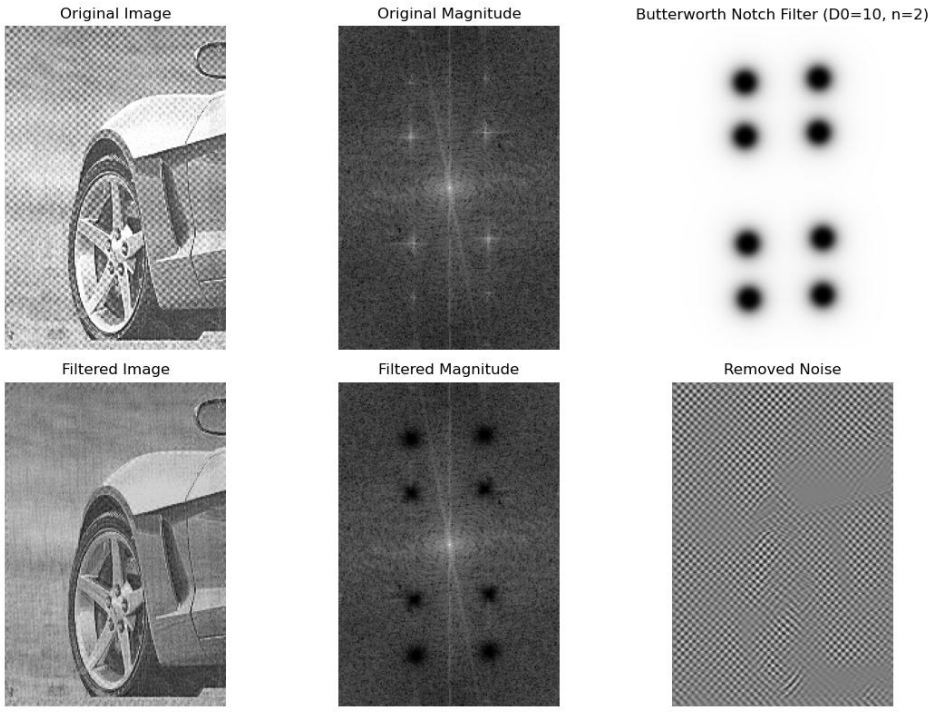
```
import numpy as np
import cv2
from matplotlib import pyplot as plt
import math

def freq_filter(image):
    image = image.astype(np.float64) / 255.0
    image_fft = np.fft.fft2(image)
    fourier = np.fft.fftshift(image_fft)
    filter = np.ones_like(image, dtype=np.complex128)
    filter[387, 475] = 0
    filter[437, 525] = 0
    filter_freq = fourier * filter
    shifted_filter_freq = np.fft.ifftshift(filter_freq)
    filter_img = np.fft.ifft2(shifted_filter_freq)
    result_img = np.abs(filter_img)
    cv2.imwrite('astronaut_filtered.png', result_img * 255)
    plt.figure(figsize=(12, 10))
    plt.subplot(2, 2, 1)
    plt.imshow(image, cmap='gray')
    plt.axis('off')
    plt.title('Original Image')
    plt.subplot(2, 2, 2)
    plt.imshow(np.abs(fourier), cmap='gray', vmin=0, vmax=5000)
    plt.axis('off')
    plt.title('Original Fourier Spectrum')
    plt.subplot(2, 2, 3)
    plt.imshow(np.abs(result_img), cmap='gray')
    plt.axis('off')
    plt.title('Result Image')
    plt.subplot(2, 2, 4)
    plt.imshow(np.abs(filter_freq), cmap='gray', vmin=0, vmax=5000)
    plt.title('Result Fourier Spectrum')
    plt.axis('off')
    plt.tight_layout()
    plt.savefig("astronaut_processed.png")

def main():
    img= cv2.imread('C:\\Users\\sty123\\Desktop\\hw\\image_processing\\HW4_2\\astronaut-
interference.tif', cv2.IMREAD_GRAYSCALE)
    freq_filter(img)

if __name__ == "__main__":
    main()
```

2. Filtered Image of “car-moire-pattern.tif”:



Source Code:

```
import numpy as np
import cv2
from matplotlib import pyplot as plt
import math

def butterworth_notch_reject(shape, centers, D0, n=2, relative_to_center=True):
    rows, cols = shape
    crow, ccol = rows // 2, cols // 2
    if relative_to_center:
        u = np.arange(rows)
        v = np.arange(cols)
        u, v = np.meshgrid(u - crow, v - ccol, indexing='ij')
    else:
        u, v = np.meshgrid(np.arange(rows), np.arange(cols), indexing='ij')
    H = np.ones((rows, cols))
    for center_u, center_v in centers:
        Dk = np.sqrt((u - center_u)**2 + (v - center_v)**2)
        if relative_to_center:
            Dk_sym = np.sqrt((u + center_u)**2 + (v + center_v)**2)
            Hk = (1 / (1 + (D0 / (Dk + 1e-6))**(2*n))) * (1 / (1 + (D0 / (Dk_sym + 1e-6))**(2*n)))
        else:
            Hk = 1 / (1 + (D0 / (Dk + 1e-6))**(2*n))
        H = H * Hk
    return H
```

```python
def butterworth_highpass_filter(shape, cutoff, order=2):
    rows, cols = shape
    crow, ccol = rows // 2, cols // 2
    u = np.arange(rows)
    v = np.arange(cols)
    u, v = np.meshgrid(u - crow, v - ccol, indexing='ij')
    D = np.sqrt(u**2 + v**2)
    H = 1 / (1 + (D / cutoff)**(2 * order))
    return H

def freq_filter(image):
    image = image.astype(np.float64) / 255.0
    image_fft = np.fft.fft2(image)
    fourier = np.fft.fftshift(image_fft)
    magnitude = 20*np.log(np.abs(fourier) + 1)  # Add 1 to avoid log(0)
    magnitude = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8UC1)
    center_row = image.shape[0] // 2
    center_col = image.shape[1] // 2
    notch_centers = [(43,55),(40,111),(84,55),(81,111),(165,57),  (161, 114), (207, 58),   (204, 114),]
    D0 = 10
    n = 2
    butterworth_filter = butterworth_notch_reject(image.shape, notch_centers, D0, n,
relative_to_center=False)
    filter_freq = fourier * butterworth_filter
    shifted_filter_freq = np.fft.ifftshift(filter_freq)
    filter_img = np.fft.ifft2(shifted_filter_freq)
    result_img = np.abs(filter_img)
    filtered_magnitude = 20*np.log(np.abs(filter_freq) + 1)
    filtered_magnitude = cv2.normalize(filtered_magnitude, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8UC1)
    filter_vis = cv2.normalize(butterworth_filter, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8UC1)
    plt.figure(figsize=(12, 8))
    plt.subplot(2, 3, 1)
    plt.imshow(image, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')
    plt.subplot(2, 3, 2)
    plt.imshow(magnitude, cmap='gray')
    plt.title('Original Magnitude')
    plt.axis('off')
    plt.subplot(2, 3, 3)
    plt.imshow(filter_vis, cmap='gray')
    plt.title(f'Butterworth Notch Filter (D0={D0}, n={n})')
    plt.axis('off')
    plt.subplot(2, 3, 4)
    plt.imshow(result_img, cmap='gray')
    plt.title('Filtered Image')
    plt.axis('off')
    plt.subplot(2, 3, 5)
    plt.imshow(filtered_magnitude, cmap='gray')
    plt.title('Filtered Magnitude')
    plt.axis('off')
    plt.subplot(2, 3, 6)
    plt.imshow(image - result_img, cmap='gray')
    plt.title('Removed Noise')
    plt.axis('off')
    plt.tight_layout()
    plt.savefig('car_butterworth_filtered.png', dpi=150, bbox_inches='tight')
    plt.show()
    cv2.imwrite('car_filtered_result.png', result_img * 255)

def main():
    img= cv2.imread('C:\\Users\\sty123\\Desktop\\hw\\image_processing\\HW4_2\\car-moire-pattern.tif',
cv2.IMREAD_GRAYSCALE)
    freq_filter(img)

if __name__ == "__main__":
    main()
```

3. We can see that for the “astronaut-interference.tif” when we change it to the frequency domain, there are 2 white dots which come from the sinusoidal noise. So, an “ideal notch filter” is designed to cut off these noise frequencies. We find the exact pixels these white dots are on and then set their values to 0; doing so eliminates the sinusoidal noise in the image.

   For the “car-moire-pattern.tif”, when we change it to the frequency domain, we can see there are many star-like shapes, which are caused by the moire pattern. To deal with this, a “Butterworth Notch Filter” is designed to cut off these frequencies by drawing a circle around the center of the stars, with a radius of 10, and setting the frequencies in this circle to 0. Doing so eliminates most of the effects of the moire pattern in the image.